# Diseño y Verificación de un Datapath de 8 Bits con Unidad de Control Microprogramada en VHDL

Aldo Mauricio Villegas Peralta[1],, Uriel Lemus Pérez[1],, Max Ramos Martínez[1],, Alfredo Ramírez Candia[1],, and Santiago Silverio Flores[1]

Equipo de Arquitectura de Computadoras,
Benemérita Universidad Autónoma de Puebla, Puebla, México
{vp202368617, lp202362797, rm202376182, rc202375947, sf202367190}@alm.buap.mx

**Abstract.** Resumen. Se diseña e implementa en VHDL-2008 un datapath de 8 bits sin signo con unidad de control basada en una ROM de microinstrucciones, siguiendo el esquema de la asignatura. El sistema se valida con 14 bancos de prueba autochequeados en GHDL y capturas en GTKWave, cubriendo cuatro experimentos: suma acumulada de 1..n, factorial, multiplicación $A \times B$ y sucesión de Fibonacci. Todos los bancos pasan; se miden los microciclos de cada caso (p. ej. $5! = 120$ en 145 microciclos, $F(13) = 233$ en 130) y se documenta una corrección del algoritmo de Fibonacci detectada por simulación.

**Palabras Clave:** Arquitectura de Computadoras, Datapath, Unidad de Control, Microprogramación, VHDL, GHDL

## 1 Introducción

La organización de un procesador se concibe habitualmente como la composición de dos subsistemas complementarios: la *ruta de datos* (datapath), que mueve y transforma los operandos a través de registros, buses y una unidad lógico-aritmética, y la *unidad de control*, que especifica en cada instante qué microoperación se ejecuta [6]. Esta separación conceptual es el hilo conductor de cualquier curso de arquitectura moderna: en las implementaciones de ciclo único de un procesador, el datapath y el control se diseñan de forma conjunta pero se describen por separado, de modo que el control se reduce a una máquina de estados que genera las señales de habilitación del datapath [14]. Una variante clásica de ese control es la *microprogramación*: en lugar de codificar la lógica de control en combinacional puro, cada estado de la máquina produce una palabra (microinstrucción) almacenada en una memoria ROM, lo que simplifica el diseño y permite inspeccionar y modificar el comportamiento sin tocar el hardware [3].

El diseño de estas estructuras se apoya en la constante reducción del coste del silicio: el número de transistores disponibles se duplica de forma periódica,

de manera que componentes que antes exigían un esfuerzo de integración enorme hoy se describen en pocas líneas de un lenguaje de descripción de hardware [22]. En este trabajo se aprovecha precisamente esa facilidad: todo el sistema se escribe en VHDL-2008 [10], un estándar formal y legible tanto por máquinas como por personas, y se verifica por simulación antes de cualquier implementación física.

El objetivo del trabajo es diseñar, implementar y verificar experimentalmente un datapath completo de 8 bits sin signo, con su unidad de control microprogramada, capaz de ejecutar cuatro algoritmos clásicos de la asignatura: la suma acumulada $S(n) = \sum_{i=1}^{n} i$, el factorial $n!$, la multiplicación por suma repetida $A \times B$ y la sucesión de Fibonacci $F(n)$. Cada algoritmo se ejecuta sobre el mismo hardware, seleccionado por un código de modo, de modo que la ROM de microinstrucciones contiene cuatro "programas" distintos. El coste de cada ejecución se cuantifica en microciclos, siguiendo la idea de medir el rendimiento de un diseño por sus ciclos [17] y los principios cuantitativos de coste y rendimiento de [8].

El artículo se organiza así. La Sección 2 describe el marco teórico y el diseño de cada módulo VHDL, incluida la ROM de microinstrucciones. La Sección 3 presenta la metodología experimental (GHDL, GTKWave y los bancos de prueba). La Sección 4 muestra las capturas de ondas de los cuatro experimentos; la Sección 5 analiza los resultados cuantitativos y cualitativos, y la Sección 6 cierra con las conclusiones y el trabajo futuro. El Apéndice A reproduce el código VHDL completo.

## 2 Diseño Teórico

### 2.1 Arquitectura general

El datapath implementa el esquema de la Fig. 1, proporcionado en la asignatura. En el centro se encuentra un *acumulador* de 8 bits que funciona como registro de propósito general del sistema: casi todas las microoperaciones leen su salida (`bus_acc`) y, cuando corresponde, capturan un nuevo valor en el flanco de reloj. Sobre él se apoyan dos caminos de selección:

- un **multiplexor superior** de cuatro entradas que alimenta la entrada del acumulador: los operandos externos (`ext1`, `ext2`), la salida del banco de registros y la realimentación del corrimiento;
- un **multiplexor inferior** de dos entradas que elige entre la salida de la ALU y la propia salida del acumulador para escribir en el banco de registros.

A la derecha del acumulador se sitúan los consumidores de `bus_acc`: la **ALU** (cuya entrada B proviene del banco de registros), el **banco de registros** de $8 \times 8$ bits con un solo puerto de lectura/escritura, y el **buffer de salida** con habilitación tri-estado que expone el resultado al exterior. A la izquierda, la salida de la ALU atraviesa el multiplexor inferior, un **corrimiento** y un **detector de cero**, y se realimenta al multiplexor superior: de este modo el resultado de cualquier operación puede volver al acumulador en el siguiente microciclo, incluidos los valores corregidos. El detector de cero genera el flag `cero` que la unidad de control usa para sus saltos condicionales.

**Table 1.** Código de operación de la ALU (campo `op_alu`).

| `op` | Operación | Descripción |
|---|---|---|
| 000 | Pasar A | $f = A$ (no nula) |
| 001 | $A + B$ | suma |
| 010 | $A - B$ | resta |
| 011 | $A + 1$ | incremento |
| 100 | $A - 1$ | decremento |
| 101 | $A \vee B$ | OR bit a bit |
| 110 | $A \wedge B$ | AND bit a bit |
| 111 | $\neg A$ | NOT de A |

Todas las operaciones son *sin signo* sobre 8 bits: los resultados se reducen módulo $2^8$, sin gestión de acarreo ni de banderas de desbordamiento [14]. Esta decisión acota el rango útil de cada algoritmo (Sección 5) pero simplifica el diseño y deja el enfoque en el control, que es el objetivo didáctico del trabajo.

### 2.2 Módulos de la ruta de datos

El diseño se descompone en diez entidades VHDL, una por fichero, todas con la lógica resoluble `std_logic` del paquete IEEE 1164 [9] y aritmética sin signo de `numeric_std` [1]. Los estilos de descripción se combinan según conviene: *dataflow* para los multiplexores y la ALU, *behavioral* para el acumulador, el banco y la control, y *estructural* para el datapath integrado, siguiendo las recomendaciones de modelado de [13] y [2].

*ALU.* La ALU de 8 bits (`alu.vhd`) recibe el operando A del acumulador y el operando B del banco de registros, y decodifica ocho operaciones con un campo de tres bits (Tabla 1). Los bloques aritméticos (suma, resta, incremento y decremento) y el corrimiento responden a los principios de diseño de datapaths aritméticos descritos en [15]; la resta se obtiene como suma del complemento a dos, con reducción módulo 256.

*Acumulador.* El acumulador (`acumulador.vhd`) es un registro de 8 bits con dos entradas de control: `wr_acc` escribe el valor del multiplexor superior y `clear_acc` reinicia a cero. Cuando ambas se activan a la vez, el *clear* tiene prioridad, convención habitual de los registros con reinicio asíncrono dominante [7].

*Banco de registros.* El banco (`banco_registro.vhd`) almacena ocho registros de 8 bits con un único puerto: la dirección `addr` selecciona qué registro se escribe (cuando `wr_rf` está activo) o qué registro se lee en `rf_dout` (en modo lectura). Esta organización de un solo puerto obliga a la microprogramación a separar en el tiempo la escritura y la lectura de un mismo registro, restricción que se refleja directamente en las tablas de microinstrucciones.

*Multiplexores, corrimiento, detector de cero y buffer.* El multiplexor superior (`mux_superior.vhd`) codifica cuatro fuentes con `sel_sup`: dos externas, el banco y la realimentación; la entrada `ext2` se deja cableada a cero por simplicidad. El multiplexor inferior (`mux_inferior.vhd`) elige entre ALU y acumulador para las escrituras en el banco. El corrimiento (`shifter.vhd`) soporta paso idéntico (`sh` $= 00$, el modo usado por los cuatro experimentos), corrimiento lógico a la izquierda y a la derecha; la señal de salida es también la que alimenta al detector de cero (`detector_cero.vhd`), de modo que el flag indica *si el valor que entra en el acumulador en el siguiente flanco será cero*, decisión tomada en la Sección 2.3. Por último, el buffer de salida (`buffer_salida.vhd`) es una compuerta tri-estado: solo cuando `out_en` está activo, `dato_out` refleja el acumulador; en caso contrario queda en alta impedancia, permitiendo compartir el bus de resultados [18], [21].

*Corrimiento barrel.* Aunque los experimentos usan el paso idéntico, el shifter está concebido como un corrimiento por N posiciones en un solo ciclo, es decir, un *barrel shifter* [20], patrón habitual en los datapaths aritméticos [15].

*Datapath integrado.* La entidad `datapath.vhd` instancia los ocho bloques anteriores por componentes, conecta el `ext_data` de entrada al multiplexor superior y expone, además de los buses, las señales de control y el flag `cero`. La instanciación por componentes (en lugar de por entidad directa) mantiene el orden de análisis independiente del orden de los ficheros, práctica recomendada para flujos de compilación por lotes [1].

### 2.3 Unidad de control

La unidad de control (`control.vhd`) es una máquina de estados [19] de un microciclo por flanco: en cada flanco de reloj avanza al siguiente paso y aplica la palabra que la ROM devuelve para ese paso. El estado se codifica en dos campos: `modo` (dos bits, selecciona el experimento) y `paso` (siete bits). La dirección de la ROM se forma como $\texttt{modo} \times 32 + \texttt{paso}$, de modo que cada experimento dispone de hasta 32 microinstrucciones contiguas, con base 0, 32, 64 y 96 para los modos 00, 01, 10 y 11 respectivamente.

Cada microinstrucción contiene los campos de la Tabla 2. Las operaciones de escritura en el acumulador, el banco y el buffer son mutuamente excluivas por construcción del codificador; el campo `dir` codifica tanto la dirección de salto incondicional (`salta`) como la del salto condicional (`salto_si_cero`), y `fin` detiene el contador de pasos y marca el final de la ejecución. La señal `ciclos` de 16 bits expone el número de microciclos transcurridos, lo que permite medir el coste de cada experimento sin instrumentación externa [17].

El salto condicional muestrea el flag `cero` *durante* el microciclo en que la ALU calcula el resultado, es decir, en el mismo flanco en que el acumulador lo captura. Esta sincronía es una decisión delicada y consciente: como el detector es combinacional, su valor deja de ser representativo justo después del flanco, cuando el acumulador ya contiene el nuevo dato. Modelar correctamente esta

**Table 2.** Campos de una microinstrucción de la ROM de control.

| Campo | Anchura | Función |
|---|---|---|
| `sel_sup` | 2 | fuente hacia el acumulador |
| `wr_acc` | 1 | escribe el acumulador |
| `clr_acc` | 1 | limpia el acumulador |
| `addr` | 3 | dirección del banco de registros |
| `wr_rf` | 1 | escribe el banco de registros |
| `op_alu` | 3 | operación de la ALU (Tabla 1) |
| `sel_inf` | 1 | 0: ALU, 1: acumulador hacia el banco |
| `sh` | 2 | modo del corrimiento |
| `out_en` | 1 | habilita el buffer de salida |
| `salta` | 1 | salto incondicional a `dir` |
| `salto_si_cero` | 1 | salta a `dir` si el flag está activo |
| `fin` | 1 | termina la ejecución |
| `dir` | 7 | dirección destino |

relación entre combinacional y secuencial es un punto central del diseño de FSM sincrónicas [12], [7], [16]; una lectura tardía del flag produciría saltos falsos. El resto de la lógica de control (decodificación, contador de pasos y banderas de fin) se describe con procesos y sentencias secuenciales al estilo de [1] y las FSM registradas de [11].

### 2.4 Algoritmos de los experimentos

Los cuatro algoritmos comparten el mismo hardware y se seleccionan con `modo`. El pseudocódigo siguiente corresponde a la secuencia de microinstrucciones efectivamente grabada en la ROM (el desglose completo de filas está en el código del Apéndice A).

*Exp. 1 — Suma de* $1 \ldots n$ *(modo 00).* $S$ se acumula en el banco y $i$ cuenta descendentemente:

```
S <- 0 ; i <- n
si i = 0 -> fin
bucle: S <- S + i ; i <- i - 1
      si i = 0 -> fin # paso la salida al buffer
      ir a bucle
```

*Exp. 2 — Factorial (modo 01).* Se mantiene el producto parcial en el banco; $k$ decrece desde $n$:

```
p <- 1 ; k <- n
si k = 0 -> fin # 0! = 1
bucle: p <- p * k ; k <- k - 1 # la multiplicacion es un
      si k = 0 -> fin # sub-bucle interno
      ir a bucle
```

*Exp. 3 — Multiplicación $A \times B$ (modo 10).* Suma repetida: se suman $B$ unidades, $A$ veces:

```
p <- 0 ; i <- A
si i = 0 -> fin # 0 x B = 0
bucle: p <- p + B ; i <- i - 1
      si i = 0 -> fin
      ir a bucle
```

*Exp. 4 — Fibonacci (modo 11).* Dos registros mantienen el par $(a, b) = (F(k), F(k+1))$; con la inicialización $a = 0$, $b = 1$, cada iteración deja $b = F(k+1)$, por lo que el bucle debe ejecutarse $n - 1$ veces para $n \geq 1$ (los casos $n = 0$ y $n = 1$ salen antes del bucle). Esta corrección fue detectada por simulación (Sección 5).

```
a <- 0 ; b <- 1
si n = 0 -> fin_a # F(0) = a
si n = 1 -> fin_b # F(1) = b
veces <- n - 1
bucle: next <- a + b ; a <- b ; b <- next
      veces <- veces - 1 ; si cero -> fin_b
      ir a bucle
fin_a: Acc <- a
fin_b: Acc <- b
```

## 3 Propuesta Experimental

La verificación se realizó por simulación con dos herramientas libres: GHDL, un simulador VHDL-2008 con analizador, elaborador y ejecutor [4], y GTKWave, un visor de ondas que lee los volcados VCD generados por GHDL [5]. Ambas se invocan exclusivamente a través de los scripts del repositorio (`scripts/vhdl-analizar`, `scripts/vhdl-sim` y `scripts/vhdl-wave`), que fijan el estándar `--std=08` y la ruta de salidas.

La estrategia de bancos de prueba *autochequeados* se aplicó en cuatro niveles, de abajo hacia arriba:

1. **Módulos (8 bancos).** Cada uno de los ocho bloques básicos tiene su banco (`tb_alu`, `tb_acumulador`, `tb_banco_registros`, `tb_mux_superior`, `tb_mux_inferior`, `tb_shifter`, `tb_detector_cero`, `tb_buffer_salida`) con `assert ... severity failure` en cada valor esperado.
2. **Integración (1 banco).** `tb_datapath` aplica microoperaciones una a una sobre el datapath completo, comprobando cada bus y el flag de cero.
3. **Control (1 banco).** `tb_control` verifica que la ROM reproduce exactamente las secuencias de los cuatro experimentos (con el flag forzado para ejercitar ambos caminos del salto) y que ningún experimento termina en 100 microciclos si el flag nunca se activa.

4. **Experimentos (4 bancos).** `tb_exp1_suma`, `tb_exp2_factorial`, `tb_exp3_mult` y `tb_exp4_fibonacci` integran datapath y control con cada modo, cargan los operandos por el puerto externo, esperan la señal `fin`, leen el resultado por `dato_out` y comparan contra el valor esperado. Además imprimen el número de microciclos (`report`) para el análisis de la Sección 5.

La matriz de pruebas se eligió para cubrir los casos límite y algunos valores intermedios: $n \in \{1, 3, 5\}$ en la suma; $n \in \{1, 5\}$ en el factorial; $(A, B) \in \{(4, 3), (5, 5), (15, 15)\}$ en la multiplicación; y $n \in \{0, 1, 7, 13\}$ en Fibonacci, incluyendo $F(0)$ (camino de salida sin bucle) y $F(13) = 233$, el mayor valor que cabe en 8 bits. El código de referencia de cada banco es el Apéndice A.

Para la sección de capturas, cada volcado VCD se cargó en GTKWave con un script Tcl que selecciona las señales de interés, aplica modo blanco y negro (contraste óptimo para impresión en escala de grises) y ajusta la vista al intervalo completo; el PNG resultante se recortó a los límites del panel de ondas con una utilidad propia incluida en el repositorio.

## 4 Simulaciones de VHDL

Las Figs. 2–5 muestran las ondas de los cuatro experimentos. En todas ellas se observan, de arriba hacia abajo: el modo seleccionado, el paso de la ROM, el acumulador, la salida del banco de registros, la salida del buffer (en alta impedancia hasta que `fin` activa `out_en`), las señales de selección de los multiplexores, el código de la ALU, la dirección del banco, la escritura del banco y el par `cero`/`fin`. Las tres "ventanas" de actividad separadas corresponden a los tres (o cuatro) casos de prueba de cada experimento.

## 5 Resultados

### 5.1 Verificación funcional

Los 14 bancos de prueba terminan sin violaciones de `assert`, de modo que se cumplen todos los objetivos funcionales: Tabla 3 resume los casos, los resultados obtenidos y los microciclos medidos. Los cuatro algoritmos producen los valores esperados, incluidos los extremos $0! = 1$, $F(0) = 0$, $F(1) = 1$ y $15 \times 15 = 225$.

### 5.2 Análisis del coste

Los microciclos permiten derivar el coste de cada algoritmo en función de su entrada, según las métricas de rendimiento por ciclos de [17] y los principios cuantitativos de [8]:

- **Suma:** los datos $(n, \text{ciclos}) = (1, 9), (3, 21), (5, 33)$ ajustan exactamente a $6n + 3$: tres microciclos de inicialización y seis por iteración del bucle, coherente con la secuencia de la ROM.

**Table 3.** Matriz de resultados: valor esperado, valor simulado y microciclos (puerto `ciclos`) de cada caso de prueba.

| Experimento | Caso | Resultado | Microciclos |
|---|---|---|---|
| Suma $1 \dots n$ | $n = 1$ | $S = 1$ | 9 |
| | $n = 3$ | $S = 6$ | 21 |
| | $n = 5$ | $S = 15$ | 33 |
| Factorial | $n = 1$ | $1! = 1$ | 10 |
| | $n = 5$ | $5! = 120$ | 145 |
| Multiplicación | $4 \times 3$ | 12 | 25 |
| | $5 \times 5$ | 25 | 37 |
| | $15 \times 15$ | 225 | 97 |
| Fibonacci | $n = 0$ | $F(0) = 0$ | 8 |
| | $n = 1$ | $F(1) = 1$ | 10 |
| | $n = 7$ | $F(7) = 13$ | 70 |
| | $n = 13$ | $F(13) = 233$ | 130 |

- **Fibonacci:** para $n \geq 1$, los datos $(1, 10), (7, 70), (13, 130)$ ajustan a $10n$: diez microciclos por iteración más la salida; $n = 0$ escapa al bucle en ocho microciclos.
- **Factorial y multiplicación:** el coste crece con la entrada pero no de forma estrictamente lineal, porque la multiplicación embebida es a su vez un subbucle cuyo número de iteraciones depende del valor de los operandos, no solo de su cantidad. Así, pasar de $4 \times 3$ a $15 \times 15$ multiplica el coste por casi cuatro, y 5! exige 145 microciclos frente a los 10 de 1!.

### 5.3 Discusión crítica

El trabajo puso de manifiesto dos cuestiones que la sola lectura del esquema no revela.

*El flag de cero es combinacional.* El detector observa la salida del corrimiento, que a su vez sale de la ALU: su valor solo es válido *durante* el microciclo, justo hasta el flanco en que el acumulador captura el resultado. Un banco de prueba que consultara el flag después del flanco observaría ya el valor recalculado con el acumulador nuevo, y en concreto $0 - 1 = 255$ en el último decremento del Exp. 1, lo que invalidaría la comprobación. Este tipo de trampas de temporización entre lógica combinacional y secuencial es justamente el contenido de los capítulos de FSM sincrónicas de [12] y [7]; el banco de integración se ajustó para muestrear el flag al final del ciclo, como hace la propia control.

*Corrección del algoritmo de Fibonacci.* La primera versión de la ROM del Exp. 4 ejecutaba $n$ iteraciones. La simulación de $n = 7$ devolvió 21, que resulta ser $F(8)$, y no $F(7) = 13$: la sucesión estaba desfasada en un término: con la inicialización $a = 0$, $b = 1$, $k$ iteraciones dejan $b = F(k + 1)$, no $F(k)$. La corrección consistió en ejecutar $n - 1$ iteraciones para $n \geq 1$, dejando $n = 0$ y $n = 1$ como caminos

de salida directa, y en ajustar la tabla del plan de trabajo. Con la corrección, la simulación confirma $F(7) = 13$ y $F(13) = 233$. Es un ejemplo tangible de cómo la verificación por simulación detecta errores de especificación que la revisión del pseudocódigo no alcanzó a ver, motivo por el cual los bancos autochequeados forman parte del flujo oficial del repositorio.

### 5.4 Límites y validez

Al trabajar sin signo y módulo 256, cada algoritmo tiene un rango de validez: la suma admite $n \leq 22$ ($S(22) = 253$); el factorial, $n \leq 5$ ($6! = 720 > 255$); la multiplicación, $A, B \leq 15$ ($16 \times 16 = 256$); y Fibonacci, $n \leq 13$, pues $F(14) = 377 > 255$. Dentro de esos rangos, los 12 casos probados cubren los extremos y confirman el comportamiento esperado; fuera de ellos, el diseño no declara desbordamiento y el resultado se reduce módulo 256, como se decidió en la Sección 2.1.

## 6 Conclusiones

Se diseñó e implementó en VHDL-2008 un datapath completo de 8 bits con unidad de control microprogramada, fiel al esquema de la asignatura, y se verificó con 14 bancos de prueba autochequeados en GHDL. Los cuatro experimentos requeridos —suma acumulada, factorial, multiplicación y Fibonacci— producen los valores correctos dentro de sus rangos de validez, y su coste en microciclos se midió y ajustó a modelos cerrados en los casos lineales ($6n + 3$ para la suma y $10n$ para Fibonacci).

La verificación experimental demostró su valor: el desfase de un término en el algoritmo de Fibonacci, invisible en la revisión del pseudocódigo, se detectó y corrigió con la simulación de $n = 7$; asimismo, el estudio del flag de cero obligó a precisar el instante de muestreo del salto condicional. Ambos hallazgos documentan el comportamiento real del diseño y quedan cubiertos por los bancos de prueba.

Como trabajo futuro se plantea: (i) ampliar la ROM para soportar operaciones con signo y detección de desbordamiento; (ii) sustituir el banco de un solo puerto por uno de dos puertos y medir la reducción de microciclos; (iii) explotar los modos de corrimiento del shifter para implementar multiplicación desplazando y sumando, en lugar de suma repetida; y (iv) estudiar la síntesis del diseño en una FPGA. Sobre este último punto conviene recordar que el GHDL disponible en el entorno del curso es una implementación *mcode*, orientada a simulación, por lo que la síntesis exigiría una herramienta adicional.

## References

1. Ashenden, P.J.: The Designer's Guide to VHDL. Morgan Kaufmann (Elsevier), San Francisco, 3rd edn. (2008), cap. 9 – sentencias secuenciales y procesos (wait, if/case), usados en el modelado sincrónico de la FSM y del banco de registros; primera guía completa de VHDL-2008 (prefacio); ISBN: 978-0-12-088785-9

2. Brown, S.D., Vranesic, Z.G.: Fundamentals of Digital Logic with VHDL Design. McGraw-Hill, New York, 3rd edn. (2009), capítulos de diseño de FSM con VHDL – ejemplos de codificación de máquinas de estado y bancos de registros usados como referencia de estilo; ISBN: 978-0-07-352953-0
3. Bryant, R.E., O'Hallaron, D.R.: Computer Systems: A Programmer's Perspective. Pearson, Boston, 3rd edn. (2015), cap. 4 (§4.1–4.4) – datapath de ciclo único y máquina de estados de la unidad de control del procesador Y86-64, análogo al par datapath/control implementado; ISBN: 978-0-13-409266-9
4. GHDL developers: GHDL: the VHDL simulator. `https://ghdl.github.io/ghdl/` (2026), secciones "Overview" y "Usage" (opciones –std=08 y –vcd); consulta: 2026-10-19
5. GTKWave developers: GTKWave analyzer. `https://gtkwave.sourceforge.net/` (2026), sección "Documentation" y guía de scripting Tcl (addSignalsFromList, Grab_To_File); consulta: 2026-10-19
6. Hamacher, V.C., Vranesic, Z.G., Zaky, S.G.: Computer Organization. McGraw-Hill, New York, 5th edn. (2002), cap. 1 – estructura básica de un computador: CPU compuesta por ruta de datos (datapath) y unidad de control, marco conceptual del informe; ISBN: 978-0-07-232086-2
7. Harris, S.L., Harris, D.M.: Digital Design and Computer Architecture: ARM Edition. Morgan Kaufmann (Elsevier), San Francisco, 1st edn. (2015), cap. 3 – diseño de máquinas de estado finito (estilo Mealy/Moore) usadas para modelar el control; cap. 4 – lenguajes de descripción de hardware con secciones de VHDL, referencia de estilo para entidades, procesos y respuestas; ISBN: 978-0-12-800056-4
8. Hennessy, J.L., Patterson, D.A.: Computer Architecture: A Quantitative Approach. Morgan Kaufmann (Elsevier), San Francisco, 6th edn. (2017), cap. 1 (§1.1–1.2) — principios cuantitativos de diseño y métricas de coste/rendimiento, aplicados aquí al coste en microciclos de cada experimento; ISBN: 978-0-12-811905-1
9. IEEE: IEEE std 1164-1993 – IEEE standard multivalue logic system for VHDL model interoperability (Std_logic_1164). Tech. rep., Institute of Electrical and Electronics Engineers (1993), paquete std_logic_1164: lógica resoluble de 9 valores usada en todos los puertos del diseño; IEEE Xplore doc. 257627; URL: https://ieeexplore.ieee.org/document/257627, consulta: 2026-10-19
10. IEEE: IEEE std 1076-2008 – IEEE standard VHDL language reference manual. Tech. rep., Institute of Electrical and Electronics Engineers (2009). `https://doi.org/10.1109/IEEESTD.2011.5967868`, definición del lenguaje VHDL-2008 implementado en los modelos (adopción ISO/IEC 61691-1-1); DOI: 10.1109/IEEESTD.2011.5967868
11. Katz, R.H., Borriello, G.: Contemporary Logic Design. Pearson Prentice Hall, Upper Saddle River, NJ, 2nd edn. (2005), capítulos de diseño secuencial – descripción de FSM y registro de estado, referencia para el desglose de la control en campos de microinstrucción; ISBN: 978-0-201-30857-0
12. Mano, M.M., Kime, C.R., Martin, T.: Logic and Computer Design Fundamentals. Pearson, Upper Saddle River, NJ, 5th edn. (2016), cap. 6 – circuitos secuenciales y máquinas de estado finito (tabla de estados y codificación), base del modelado de la unidad de control; ISBN: 978-0-13-3760637
13. Navabi, Z.: VHDL: Analysis and Modeling of Digital Systems. McGraw-Hill, New York, 2nd edn. (1998), capítulos de modelado – estilos de descripción (dataflow, behavioral, structural) combinados en la jerarquía del datapath (§2.2 del informe); ISBN: 978-0-07-046479-7

14. Patterson, D.A., Hennessy, J.L.: Computer Organization and Design: The Hardware/Software Interface (RISC-V Edition). Morgan Kaufmann (Elsevier), Amsterdam, 5th edn. (2017), cap. 1 (§1.1) — abstracción por capas del hardware; cap. 4 (§4.3–4.4) – separación datapath / unidad de control en una implementación de ciclo único, usada como marco de la arquitectura de la Fig. 1; ISBN: 978-0-12-812275-4
15. Rabaey, J.M., Chandrakasan, A., Nikolić, B.: Digital Integrated Circuits: A Design Perspective. Pearson Prentice Hall, Upper Saddle River, NJ, 2nd edn. (2003), cap. 11 (§11.1–11.4) – datapaths de procesadores digitales, sumadores, multiplicadores y corrimientos: fundamentos de la ALU, del corrimiento iterativo de la multiplicación y del barrel shifter; ISBN: 978-0-13-090996-1
16. Roth, C.H., Kinney, L.L.: Fundamentals of Logic Design. Cengage Learning, Stamford, CT, 7th edn. (2013), unidades de máquinas de estado finito – tablas de transición y diagramas de estados, referencia del desglose de la FSM de control; ISBN: 978-1-133-62847-7
17. Stallings, W.: Computer Organization and Architecture: Designing for Performance. Pearson, Upper Saddle River, NJ, 10th edn. (2015), cap. 2 – métricas de rendimiento (tiempo de ejecución y ciclos), aplicadas al coste en microciclos reportado en la Tabla 3; ISBN: 978-0-13-410161-3
18. Wakerly, J.F.: Digital Design Principles and Practices. Prentice Hall (Pearson), Upper Saddle River, NJ, 4th edn. (2005), cap. 8 – máquinas de estado finito: codificación de estados y salidas Moore/Mealy; sección de buffers tri-estado – salida de bus compartido del buffer de salida; ISBN: 978-0-13-186389-7
19. Wikipedia contributors: Autómata finito. `https://es.wikipedia.org/wiki/Aut%C3%B3mata_finito` (2026), sección "Definición formal" – quintuple (estados, entrada, transición, estado inicial, estados finales); consulta: 2026-10-19
20. Wikipedia contributors: Barrel shifter. `https://en.wikipedia.org/wiki/Barrel_shifter` (2026), sección "Overview" – corrimiento por N posiciones en un solo ciclo; consulta: 2026-10-19
21. Wikipedia contributors: Buffer tri-estado. `https://es.wikipedia.org/wiki/Buffer_tri-estado` (2026), sección "Descripción" – salida en alta impedancia (Z) para compartir un bus; consulta: 2026-10-19
22. Wikipedia contributors: Moore's law. `https://es.wikipedia.org/wiki/Ley_de_Moore` (2026), sección "Enunciado" – doblecimiento periódico del número de transistores; consulta: 2026-10-19

## A Código VHDL

Código completo de la unidad, tomado directamente de `vhdl/datapath-3/src/` mediante `lstinputlisting`, de modo que el apéndice permanece sincronizado con el repositorio.

**Listing 1.1.** alu.vhd — unidad lógico-aritmética de 8 bits (Tabla 1)

```
-- ============================================================
-- ALU del Datapath 3
-- 8 operaciones aritmético-lógicas sobre bytes sin signo,
-- seleccionadas por A2 A1 A0 (tabla de la asignatura).
-- A = salida del acumulador, B = salida del banco de registros.
-- Combinacional. Sin flag de acarreo (limitación documentada:
-- los desbordamientos son silenciosos, módulo 256).
-- ============================================================
```

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity alu is
    port (
        a  : in  std_logic_vector(7 downto 0);
        b  : in  std_logic_vector(7 downto 0);
        op : in  std_logic_vector(2 downto 0);  -- A2 A1 A0
        f  : out std_logic_vector(7 downto 0)
    );
end entity;

architecture comportamental of alu is
begin
    process (a, b, op)
    begin
        case op is
            when "000" =>
                f <= a;                                                    --
                    Pasar A
            when "001" =>
                f <= std_logic_vector(unsigned(a) + unsigned(b));          -- Suma
            when "010" =>
                f <= std_logic_vector(unsigned(a) - unsigned(b));          --
                    Resta
            when "011" =>
                f <= std_logic_vector(unsigned(a) + 1);                    --
                    Incremento
            when "100" =>
                f <= std_logic_vector(unsigned(a) - 1);                    --
                    Decremento
            when "101" =>
                f <= a or b;                                               -- OR
            when "110" =>
                f <= a and b;                                              -- AND
            when others =>
                f <= not a;                                                -- NOT
                    A (111)
        end case;
    end process;
end architecture;
```

**Listing 1.2.** acumulador.vhd — registro acumulador con escritura y clear

```
-- ============================================================
-- Acumulador del Datapath 3
-- Registro de 8 bits con escritura (wr) y limpieza síncrona
-- (clear, con prioridad sobre wr) más reset global. Almacena
-- operandos y resultados parciales; su salida es el bus_acc.
-- ============================================================
library ieee;
use ieee.std_logic_1164.all;

entity acumulador is
    port (
        clk   : in  std_logic;
        rst   : in  std_logic;
        wr    : in  std_logic;
        clear : in  std_logic;
        d_in  : in  std_logic_vector(7 downto 0);
        q     : out std_logic_vector(7 downto 0)
    );
end entity;

architecture comportamental of acumulador is
    signal registro : std_logic_vector(7 downto 0) := (others => '0');
```

```
begin
    process (clk)
    begin
        if rising_edge(clk) then
            if rst = '1' then
                registro <= (others => '0');
            elsif clear = '1' then
                registro <= (others => '0');
            elsif wr = '1' then
                registro <= d_in;
            end if;
        end if;
    end process;

    q <= registro;
end architecture;
```

**Listing 1.3.** banco_registros.vhd — banco de 8 registros de 8 bits con un solo puerto

```
-- ============================================================
-- Banco de registros del Datapath 3
-- 8 registros de 8 bits. Lectura combinacional en "addr";
-- escritura síncrona de bus_acc en RF[addr] cuando wr = '1'
-- (un solo puerto: en cada ciclo se lee y escribe la misma
-- dirección, como en el diagrama de la asignatura).
-- ============================================================
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity banco_registros is
    port (
        clk    : in  std_logic;
        rst    : in  std_logic;
        addr   : in  std_logic_vector(2 downto 0);
        wr     : in  std_logic;
        d_in   : in  std_logic_vector(7 downto 0);
        d_out  : out std_logic_vector(7 downto 0)
    );
end entity;

architecture comportamental of banco_registros is
    type banco_t is array (0 to 7) of std_logic_vector(7 downto 0);
    signal regs : banco_t := (others => (others => '0'));
begin
    process (clk)
    begin
        if rising_edge(clk) then
            if rst = '1' then
                regs <= (others => (others => '0'));
            elsif wr = '1' then
                regs(to_integer(unsigned(addr))) <= d_in;
            end if;
        end if;
    end process;

    d_out <= regs(to_integer(unsigned(addr)));
end architecture;
```

**Listing 1.4.** mux_superior.vhd — multiplexor de cuatro fuentes hacia el acumulador

```
-- ============================================================
-- Mux superior del Datapath 3 (4 -> 1)
-- Elige el dato que se carga en el acumulador.
-- sel_sup: 00 = EXT1 (dato externo), 01 = EXT2 (reservado),
--          10 = salida del banco de registros,
```

```
--          11 = realimentación del shifter.
-- Códigos simbólicos: confirmar con el diagrama oficial.
-- ============================================================
library ieee;
use ieee.std_logic_1164.all;

entity mux_superior is
    port (
        sel_sup : in  std_logic_vector(1 downto 0);
        ext1    : in  std_logic_vector(7 downto 0);
        ext2    : in  std_logic_vector(7 downto 0);
        rf      : in  std_logic_vector(7 downto 0);
        sh      : in  std_logic_vector(7 downto 0);
        y       : out std_logic_vector(7 downto 0)
    );
end entity;

architecture comportamental of mux_superior is
begin
    with sel_sup select y <=
        ext1 when "00",
        ext2 when "01",
        rf   when "10",
        sh   when others;
end architecture;
```

**Listing 1.5.** mux_inferior.vhd — multiplexor entre ALU y acumulador

```
-- ============================================================
-- Mux inferior del Datapath 3 (2 -> 1)
-- Selecciona qué dato entra al shifter.
-- sel_inf: 0 = salida de la ALU, 1 = bus_acc (acumulador).
-- Códigos simbólicos: confirmar con el diagrama oficial.
-- ============================================================
library ieee;
use ieee.std_logic_1164.all;

entity mux_inferior is
    port (
        sel_inf : in  std_logic;
        alu_f   : in  std_logic_vector(7 downto 0);
        acc     : in  std_logic_vector(7 downto 0);
        y       : out std_logic_vector(7 downto 0)
    );
end entity;

architecture comportamental of mux_inferior is
begin
    with sel_inf select y <=
        alu_f when '0',
        acc   when others;
end architecture;
```

**Listing 1.6.** shifter.vhd — corrimiento con paso idéntico (usado por los experimentos)

```
-- ============================================================
-- Shifter del Datapath 3
-- Desplazamiento de 1 bit sobre el dato de entrada. Su salida
-- realimenta al mux superior (camino obligatorio del resultado
-- hacia el acumulador), por lo que el modo "pasar" (sh = 00)
-- es imprescindible.
-- sh = 00: pasar (idéntico)        -- IMPRESCINDIBLE (confirmado en uso)
-- sh = 01: desplazamiento izquierda, rellena con 0   -- PROVISIONAL
-- sh = 10: desplazamiento derecha lógico, rellena 0  -- PROVISIONAL
-- sh = 11: desplazamiento derecha aritmético         -- PROVISIONAL
-- Los códigos 01/10/11 son provisionales: el diagrama de la
```

```
-- asignatura no documenta dirección ni códigos del shifter;
-- los 4 experimentos solo usan sh = 00.
-- ============================================================
library ieee;
use ieee.std_logic_1164.all;

entity shifter is
    port (
        d_in  : in  std_logic_vector(7 downto 0);
        sh    : in  std_logic_vector(1 downto 0);
        d_out : out std_logic_vector(7 downto 0)
    );
end entity;

architecture comportamental of shifter is
begin
    with sh select d_out <=
        d_in                            when "00",  -- pasar
        d_in(6 downto 0) & '0'          when "01",  -- << 1 (provisional)
        '0' & d_in(7 downto 1)          when "10",  -- >> 1 lógico (provisional)
        d_in(7) & d_in(7 downto 1)      when others; -- >> 1 aritmético (
            provisional)
end architecture;
```

**Listing 1.7.** detector_cero.vhd — flag de cero sobre la salida del corrimiento

```
-- ============================================================
-- Detector de cero del Datapath 3
-- Añadido con justificación (no aparece en el diagrama): sin
-- una condición de "contador = 0" ningún algoritmo iterativo
-- puede terminar. Observa la salida del shifter (el valor que
-- se escribirá en el acumulador en el mismo flanco), de modo
-- que el salto condicional de la unidad de control puede ir en
-- la misma microinstrucción que produce el resultado.
-- ============================================================
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity detector_cero is
    port (
        d    : in  std_logic_vector(7 downto 0);
        cero : out std_logic
    );
end entity;

architecture comportamental of detector_cero is
begin
    cero <= '1' when unsigned(d) = 0 else '0';
end architecture;
```

**Listing 1.8.** buffer_salida.vhd — buffer tri-estado de resultados

```
-- ============================================================
-- Buffer de salida del Datapath 3
-- Triángulo inferior izquierdo del diagrama: saca bus_acc al
-- puerto dato_out cuando está habilitado (en = '1'); en caso
-- contrario la salida queda en alta impedancia (tri-estado).
-- Interpretación provisional: si el diagrama oficial es un
-- buffer simple (sin habilitación), basta en = '1' siempre.
-- ============================================================
library ieee;
use ieee.std_logic_1164.all;

entity buffer_salida is
    port (
```

```
        d_in : in  std_logic_vector(7 downto 0);
        en   : in  std_logic;
        y    : out std_logic_vector(7 downto 0)
    );
end entity;

architecture comportamental of buffer_salida is
begin
    y <= d_in when en = '1' else (others => 'Z');
end architecture;
```

**Listing 1.9.** datapath.vhd — integración estructural de los ocho módulos

```
-- ============================================================
-- Datapath 3: integración estructural del diagrama completo
-- mux superior → acumulador → (banco de registros | ALU) →
-- mux inferior → shifter → realimentación al mux superior,
-- más el detector de cero y el buffer de salida.
--
-- Se usa instanciación por componentes para que el análisis
-- sea independiente del orden de los ficheros (scripts/vhdl-analizar).
--
-- Señales de entrada: las de control generadas por control.vhd.
-- Salidas observables: cero (flag), dato_out (puerto externo),
-- acc_q y rf_dout (para testbenches y capturas de ondas).
-- ============================================================
library ieee;
use ieee.std_logic_1164.all;

entity datapath is
    port (
        clk      : in  std_logic;
        rst      : in  std_logic;
        ext_data : in  std_logic_vector(7 downto 0);  -- EXT1
        sel_sup  : in  std_logic_vector(1 downto 0);
        wr_acc   : in  std_logic;
        clr_acc  : in  std_logic;
        addr     : in  std_logic_vector(2 downto 0);
        wr_rf    : in  std_logic;
        op_alu   : in  std_logic_vector(2 downto 0);
        sel_inf  : in  std_logic;
        sh       : in  std_logic_vector(1 downto 0);
        out_en   : in  std_logic;
        cero     : out std_logic;
        dato_out : out std_logic_vector(7 downto 0);
        acc_q    : out std_logic_vector(7 downto 0);
        rf_dout  : out std_logic_vector(7 downto 0)
    );
end entity;

architecture estructural of datapath is
    component mux_superior is
        port (
            sel_sup : in  std_logic_vector(1 downto 0);
            ext1    : in  std_logic_vector(7 downto 0);
            ext2    : in  std_logic_vector(7 downto 0);
            rf      : in  std_logic_vector(7 downto 0);
            sh      : in  std_logic_vector(7 downto 0);
            y       : out std_logic_vector(7 downto 0)
        );
    end component;

    component acumulador is
        port (
            clk   : in  std_logic;
            rst   : in  std_logic;
            wr    : in  std_logic;
```

```
        clear : in  std_logic;
        d_in  : in  std_logic_vector(7 downto 0);
        q     : out std_logic_vector(7 downto 0)
    );
end component;

component banco_registros is
    port (
        clk   : in  std_logic;
        rst   : in  std_logic;
        addr  : in  std_logic_vector(2 downto 0);
        wr    : in  std_logic;
        d_in  : in  std_logic_vector(7 downto 0);
        d_out : out std_logic_vector(7 downto 0)
    );
end component;

component alu is
    port (
        a  : in  std_logic_vector(7 downto 0);
        b  : in  std_logic_vector(7 downto 0);
        op : in  std_logic_vector(2 downto 0);
        f  : out std_logic_vector(7 downto 0)
    );
end component;

component mux_inferior is
    port (
        sel_inf : in  std_logic;
        alu_f   : in  std_logic_vector(7 downto 0);
        acc     : in  std_logic_vector(7 downto 0);
        y       : out std_logic_vector(7 downto 0)
    );
end component;

component shifter is
    port (
        d_in  : in  std_logic_vector(7 downto 0);
        sh    : in  std_logic_vector(1 downto 0);
        d_out : out std_logic_vector(7 downto 0)
    );
end component;

component detector_cero is
    port (
        d    : in  std_logic_vector(7 downto 0);
        cero : out std_logic
    );
end component;

component buffer_salida is
    port (
        d_in : in  std_logic_vector(7 downto 0);
        en   : in  std_logic;
        y    : out std_logic_vector(7 downto 0)
    );
end component;

-- buses internos (replica del diagrama)
signal ext2_nulo : std_logic_vector(7 downto 0) := (others => '0');
signal mux_sup_y : std_logic_vector(7 downto 0);  -- hacia la entrada del
     acumulador
signal bus_acc   : std_logic_vector(7 downto 0);  -- salida del acumulador (el
      "bus")
signal rf_out    : std_logic_vector(7 downto 0);
signal alu_f     : std_logic_vector(7 downto 0);
signal mux_inf_y : std_logic_vector(7 downto 0);
signal sh_out    : std_logic_vector(7 downto 0);
```

```
begin
    u_mux_sup : mux_superior port map (
        sel_sup => sel_sup,
        ext1    => ext_data,
        ext2    => ext2_nulo,
        rf      => rf_out,
        sh      => sh_out,
        y       => mux_sup_y
    );

    u_acumulador : acumulador port map (
        clk   => clk,
        rst   => rst,
        wr    => wr_acc,
        clear => clr_acc,
        d_in  => mux_sup_y,
        q     => bus_acc
    );

    u_banco : banco_registros port map (
        clk   => clk,
        rst   => rst,
        addr  => addr,
        wr    => wr_rf,
        d_in  => bus_acc,
        d_out => rf_out
    );

    u_alu : alu port map (
        a  => bus_acc,
        b  => rf_out,
        op => op_alu,
        f  => alu_f
    );

    u_mux_inf : mux_inferior port map (
        sel_inf => sel_inf,
        alu_f   => alu_f,
        acc     => bus_acc,
        y       => mux_inf_y
    );

    u_shifter : shifter port map (
        d_in  => mux_inf_y,
        sh    => sh,
        d_out => sh_out
    );

    u_detector : detector_cero port map (
        d    => sh_out,
        cero => cero
    );

    u_buffer : buffer_salida port map (
        d_in => bus_acc,
        en   => out_en,
        y    => dato_out
    );

    -- salidas de observación (testbenches y capturas)
    acc_q   <= bus_acc;
    rf_dout <= rf_out;
end architecture;
```

**Listing 1.10.** control.vhd — unidad de control con ROM de microinstrucciones de los cuatro experimentos

```
-- ============================================================
-- Unidad de control del Datapath 3 (FSM secuencial)
-- Genera las señales de control del datapath a partir de una
-- ROM de microinstrucciones: la tabla de microoperaciones del
-- plan (docs/plan-datapath-3.md, §8) es literalmente el
-- contenido de la ROM.
--
--   * paso (7 bits) = dirección de microcódigo activa;
--     base = modo * 32  (00=exp1, 01=exp2, 10=exp3, 11=exp4)
--   * un microciclo por flanco de subida;
--   * salto incondicional (salta) o condicional (salto_si_cero
--     con el flag "cero" del datapath);
--   * fin = última palabra del algoritmo; el contador se detiene;
--   * ciclos = microciclos ejecutados hasta alcanzar "fin".
--
-- "modo" debe mantenerse estable durante la ejecución.
-- ============================================================
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity control is
    port (
        clk        : in  std_logic;
        rst        : in  std_logic;
        cero       : in  std_logic;
        modo       : in  std_logic_vector(1 downto 0);
        sel_sup    : out std_logic_vector(1 downto 0);
        wr_acc     : out std_logic;
        clr_acc    : out std_logic;
        addr       : out std_logic_vector(2 downto 0);
        wr_rf      : out std_logic;
        op_alu     : out std_logic_vector(2 downto 0);
        sel_inf    : out std_logic;
        sh         : out std_logic_vector(1 downto 0);
        out_en     : out std_logic;
        fin        : out std_logic;
        paso       : out std_logic_vector(6 downto 0);
        ciclos     : out std_logic_vector(15 downto 0)
    );
end entity;

architecture fsm of control is
    type palabra_t is record
        sel_sup       : std_logic_vector(1 downto 0);
        wr_acc        : std_logic;
        clr_acc       : std_logic;
        addr          : std_logic_vector(2 downto 0);
        wr_rf         : std_logic;
        op_alu        : std_logic_vector(2 downto 0);
        sel_inf       : std_logic;
        sh            : std_logic_vector(1 downto 0);
        out_en        : std_logic;
        salta         : std_logic;
        salto_si_cero : std_logic;
        dir           : natural range 0 to 127;
        fin           : std_logic;
    end record;

    type rom_t is array (0 to 127) of palabra_t;

    -- constructor de palabras: los campos no indicados toman su
    -- valor por defecto (sin escritura, sin salto, op = pasar)
    function mk(
        sel_sup       : std_logic_vector(1 downto 0) := "00";
        wr_acc        : std_logic := '0';
        clr_acc       : std_logic := '0';
        addr          : std_logic_vector(2 downto 0) := "000";
```

```
        wr_rf         : std_logic := '0';
        op_alu        : std_logic_vector(2 downto 0) := "000";
        sel_inf       : std_logic := '0';
        sh            : std_logic_vector(1 downto 0) := "00";
        out_en        : std_logic := '0';
        salta         : std_logic := '0';
        salto_si_cero : std_logic := '0';
        dir           : natural := 0;
        fin           : std_logic := '0'
    ) return palabra_t is
        variable p : palabra_t;
    begin
        p.sel_sup       := sel_sup;
        p.wr_acc        := wr_acc;
        p.clr_acc       := clr_acc;
        p.addr          := addr;
        p.wr_rf         := wr_rf;
        p.op_alu        := op_alu;
        p.sel_inf       := sel_inf;
        p.sh            := sh;
        p.out_en        := out_en;
        p.salta         := salta;
        p.salto_si_cero := salto_si_cero;
        p.dir           := dir;
        p.fin           := fin;
        return p;
    end function;

    -- ROM de microinstrucciones (tabla §8 del plan)
    constant ROM : rom_t := (
        ----------------------------------------------------------------
        -- Exp 1: suma de 1..n (pasos 0..10) — plan §8.1
        ----------------------------------------------------------------
        0  => mk(clr_acc => '1'),
        1  => mk(wr_acc => '1'),                                    -- Acc ← n (EXT1
          )
        2  => mk(wr_rf => '1', addr => "001"),                      -- R1 ← Acc
        3  => mk(clr_acc => '1'),                                   -- s = 0
        4  => mk(wr_acc => '1', sel_sup => "10", addr => "010"),    -- Acc ← s
        5  => mk(wr_acc => '1', sel_sup => "11", op_alu => "001",
                 addr => "001"),                                    -- Acc ← s + i
        6  => mk(wr_rf => '1', addr => "010"),                      -- R2 ← Acc
        7  => mk(wr_acc => '1', sel_sup => "10", addr => "001"),    -- Acc ← i
        8  => mk(wr_acc => '1', sel_sup => "11", op_alu => "100",
                 salto_si_cero => '1', dir => 10),                  -- Acc ← i-1
        9  => mk(wr_rf => '1', addr => "001", salta => '1', dir => 4),
        10 => mk(wr_acc => '1', sel_sup => "10", addr => "010",
                 out_en => '1', fin => '1'),                        -- Acc ← s (fin)
        ----------------------------------------------------------------
        -- Exp 2: factorial de n (pasos 32..63) — plan §8.2
        ----------------------------------------------------------------
        32 => mk(clr_acc => '1'),
        33 => mk(wr_acc => '1'),                                    -- Acc ← 1 (EXT1
          )
        34 => mk(wr_rf => '1', addr => "010"),                      -- fact = 1
        35 => mk(wr_acc => '1'),                                    -- Acc ← n (EXT1
          )
        36 => mk(wr_acc => '1', sel_sup => "11", op_alu => "100"),-- Acc ← n-1
        37 => mk(wr_rf => '1', addr => "101"),                      -- veces = n-1
        38 => mk(wr_acc => '1'),                                    -- Acc ← 2 (EXT1
          )
        39 => mk(wr_rf => '1', addr => "001"),                      -- i = 2
        40 => mk(clr_acc => '1'),
        41 => mk(wr_acc => '1', sel_sup => "11", op_alu => "010",
                 addr => "101", salto_si_cero => '1', dir => 63), -- ¿n = 1?
        42 => mk(wr_acc => '1', sel_sup => "10", addr => "010"),    -- X = fact
        43 => mk(wr_rf => '1', addr => "011"),
        44 => mk(wr_acc => '1', sel_sup => "10", addr => "001"),    -- Y = i
```

```
45 => mk(wr_rf => '1', addr => "100"),
46 => mk(clr_acc => '1'),
47 => mk(wr_rf => '1', addr => "110"),                    -- P = 0
48 => mk(wr_acc => '1', sel_sup => "10", addr => "110"),  -- P (bucle)
49 => mk(wr_acc => '1', sel_sup => "11", op_alu => "001",
        addr => "011"),                                   -- P + X
50 => mk(wr_rf => '1', addr => "110"),
51 => mk(wr_acc => '1', sel_sup => "10", addr => "100"),  -- Y
52 => mk(wr_acc => '1', sel_sup => "11", op_alu => "100",
        salto_si_cero => '1', dir => 55),                 -- Y-1
53 => mk(wr_rf => '1', addr => "100", salta => '1', dir => 48),
54 => mk(salta => '1', dir => 48),                        -- (no alcanzado
    )
55 => mk(wr_acc => '1', sel_sup => "10", addr => "110"),  -- producto
56 => mk(wr_rf => '1', addr => "010"),                    -- fact ← P
57 => mk(wr_acc => '1', sel_sup => "10", addr => "001"),  -- i
58 => mk(wr_acc => '1', sel_sup => "11", op_alu => "011"),-- i+1
59 => mk(wr_rf => '1', addr => "001"),
60 => mk(wr_acc => '1', sel_sup => "10", addr => "101"),  -- veces
61 => mk(wr_acc => '1', sel_sup => "11", op_alu => "100",
        salto_si_cero => '1', dir => 63),                 -- veces-1
62 => mk(wr_rf => '1', addr => "101", salta => '1', dir => 42),
63 => mk(wr_acc => '1', sel_sup => "10", addr => "010",
        out_en => '1', fin => '1'),                       -- n! (fin)
-------------------------------------------------------------------
-- Exp 3: multiplicación A x B (pasos 64..78) — plan §8.3
-------------------------------------------------------------------
64 => mk(clr_acc => '1'),
65 => mk(wr_acc => '1'),                                  -- Acc ← A (EXT1
    )
66 => mk(wr_rf => '1', addr => "011"),
67 => mk(wr_acc => '1'),                                  -- Acc ← B (EXT1
    )
68 => mk(wr_rf => '1', addr => "100"),
69 => mk(clr_acc => '1'),
70 => mk(wr_rf => '1', addr => "110"),                    -- P = 0
71 => mk(wr_acc => '1', sel_sup => "11", op_alu => "010",
        addr => "100", salto_si_cero => '1', dir => 78), -- ¿B = 0?
72 => mk(wr_acc => '1', sel_sup => "10", addr => "110"),  -- P (bucle)
73 => mk(wr_acc => '1', sel_sup => "11", op_alu => "001",
        addr => "011"),                                   -- P + A
74 => mk(wr_rf => '1', addr => "110"),
75 => mk(wr_acc => '1', sel_sup => "10", addr => "100"),  -- B'
76 => mk(wr_acc => '1', sel_sup => "11", op_alu => "100",
        salto_si_cero => '1', dir => 78),                 -- B'-1
77 => mk(wr_rf => '1', addr => "100", salta => '1', dir => 72),
78 => mk(wr_acc => '1', sel_sup => "10", addr => "110",
        out_en => '1', fin => '1'),                       -- A x B (fin)
-------------------------------------------------------------------
-- Exp 4: Fibonacci F(n) (pasos 96..118) — plan §8.4
-- Con a=0, b=1, k iteraciones dejan b = F(k+1); por eso el
-- bucle se ejecuta n-1 veces (y n = 0 / n = 1 salen antes).
-------------------------------------------------------------------
96  => mk(clr_acc => '1'),
97  => mk(wr_rf => '1', addr => "010"),                  -- a = 0
98  => mk(wr_acc => '1'),                                -- Acc ← 1 (EXT1
    )
99  => mk(wr_rf => '1', addr => "011"),                  -- b = 1
100 => mk(wr_acc => '1'),                                -- Acc ← n (EXT1
    )
101 => mk(wr_rf => '1', addr => "001"),
102 => mk(clr_acc => '1'),
103 => mk(wr_acc => '1', sel_sup => "11", op_alu => "010",
         addr => "001", salto_si_cero => '1', dir => 117), -- ¿n = 0?
104 => mk(wr_acc => '1', sel_sup => "10", addr => "001"), -- Acc ← n
105 => mk(wr_acc => '1', sel_sup => "11", op_alu => "100",
         salto_si_cero => '1', dir => 118),               -- ¿n = 1?
106 => mk(wr_rf => '1', addr => "001"),                  -- R1 = n-1
```

```
        107 => mk(wr_acc => '1', sel_sup => "10", addr => "010"), -- a
        108 => mk(wr_acc => '1', sel_sup => "11", op_alu => "001",
                  addr => "011"),                                  -- next = a+b
        109 => mk(wr_rf => '1', addr => "100"),
        110 => mk(wr_acc => '1', sel_sup => "10", addr => "011"), -- b
        111 => mk(wr_rf => '1', addr => "010"),                   -- a' = b
        112 => mk(wr_acc => '1', sel_sup => "10", addr => "100"), -- next
        113 => mk(wr_rf => '1', addr => "011"),                   -- b' = next
        114 => mk(wr_acc => '1', sel_sup => "10", addr => "001"), -- R1
        115 => mk(wr_acc => '1', sel_sup => "11", op_alu => "100",
                  salto_si_cero => '1', dir => 118),              -- R1-1
        116 => mk(wr_rf => '1', addr => "001", salta => '1', dir => 107),
        117 => mk(wr_acc => '1', sel_sup => "10", addr => "010",
                  out_en => '1', fin => '1'),                     -- F(0) (fin)
        118 => mk(wr_acc => '1', sel_sup => "10", addr => "011",
                  out_en => '1', fin => '1'),                     -- F(n) (fin)
        -- cualquier otra dirección: palabra fin (falla pronto y se ve)
        others => mk(fin => '1')
    );

    signal paso_reg   : unsigned(6 downto 0) := (others => '0');
    signal ciclos_reg : unsigned(15 downto 0) := (others => '0');
    signal palabra    : palabra_t;
begin
    palabra <= ROM(to_integer(paso_reg));

    sel_sup <= palabra.sel_sup;
    wr_acc  <= palabra.wr_acc;
    clr_acc <= palabra.clr_acc;
    addr    <= palabra.addr;
    wr_rf   <= palabra.wr_rf;
    op_alu  <= palabra.op_alu;
    sel_inf <= palabra.sel_inf;
    sh      <= palabra.sh;
    out_en  <= palabra.out_en;
    fin     <= palabra.fin;
    paso    <= std_logic_vector(paso_reg);
    ciclos  <= std_logic_vector(ciclos_reg);

    process (clk)
    begin
        if rising_edge(clk) then
            if rst = '1' then
                paso_reg   <= unsigned(modo & "00000");  -- base = modo * 32
                ciclos_reg <= (others => '0');
            elsif palabra.fin = '0' then
                if palabra.salto_si_cero = '1' and cero = '1' then
                    paso_reg <= to_unsigned(palabra.dir, 7);
                elsif palabra.salta = '1' then
                    paso_reg <= to_unsigned(palabra.dir, 7);
                else
                    paso_reg <= paso_reg + 1;
                end if;
                ciclos_reg <= ciclos_reg + 1;
            end if;
        end if;
    end process;
end architecture;
```

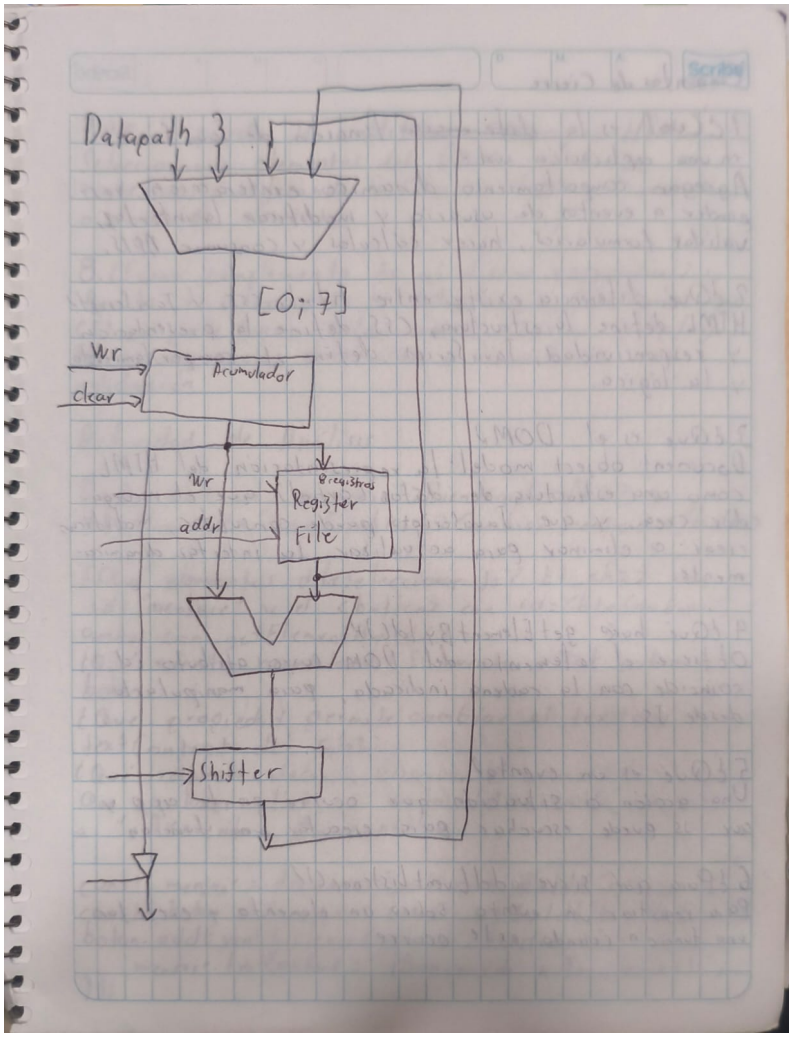



**Fig. 1.** Esquema del datapath proporcionado en la asignatura. Se observan el acumulador central, el multiplexor superior de cuatro entradas, el banco de registros, la ALU, el multiplexor inferior, el corrimiento con detección de cero y el buffer de salida.

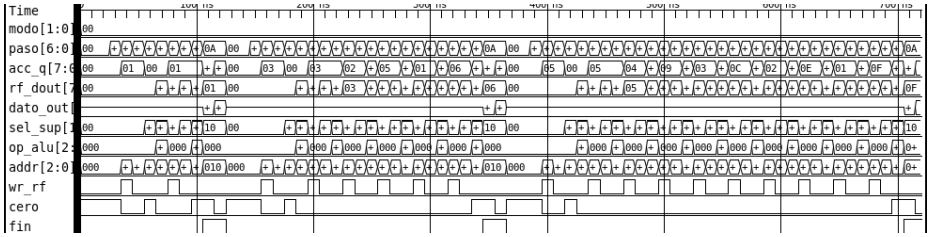



**Fig. 2.** Experimento 1 — Suma de $1 \ldots n$ (modo 00). Casos $n = 1$, $n = 3$ y $n = 5$. Se observa el bucle de seis microciclos por iteración y la salida del resultado en `dato_out` al final de cada caso.

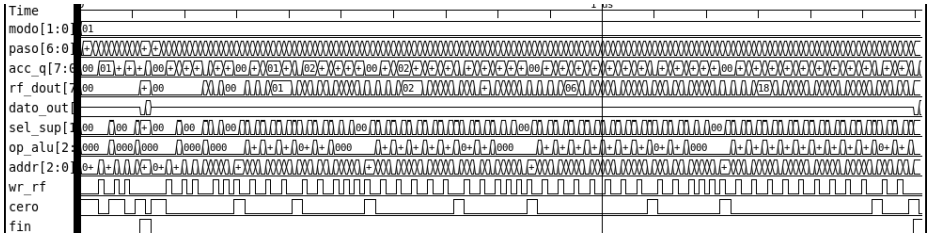



**Fig. 3.** Experimento 2 — Factorial (modo 01). Casos $n = 1$ y $n = 5$. La actividad densa corresponde al sub-bucle de multiplicación por suma repetida embebido en cada iteración del factorial.

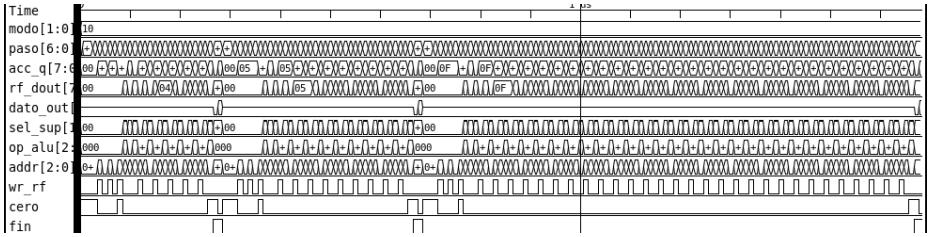



**Fig. 4.** Experimento 3 — Multiplicación $A \times B$ (modo 10). Casos $(4, 3)$, $(5, 5)$ y $(15, 15)$. El bucle exterior recorre $A$ iteraciones de suma de $B$.

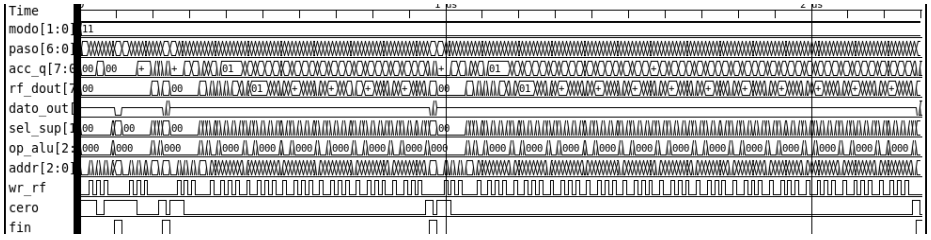



**Fig. 5.** Experimento 4 — Fibonacci (modo 11). Casos $n = 0$, $n = 1$, $n = 7$ y $n = 13$. Los dos primeros terminan antes de entrar al bucle; los restantes ejecutan $n - 1$ iteraciones.